```
// Token aggiunti più avanti dentro la funzione `main()` dopo il deploy dello
Swapper.
// Tutte le chiamate await swapper.addSupportedToken devono essere all'interno
di async function main()
"use strict";
require('dotenv').config({ path: process.env.HARDHAT_NETWORK === 'localhost' ?
'.env.local' : '.env' });
const { ethers, upgrades } = require('hardhat');
const fs = require('fs');

async function main() {
  console.log("🚀 DEPLOY MINIMAL NFT/FT CONTRACTS ON BASE");
  console.log("═══════════════════════════════════════════════════");
  // DEBUG SNIPPET: Diagnosi ruoli e permessi deployer su CosmixProtocolToken
(FT)
  console.log("== DEBUG: Diagnostica ruoli su FT ==");
  const minterRoleHash = await ft.MINTER_ROLE();    // bytes32 hash -
COSMIX_MINTER_ROLE
  const managerRoleHash = await ft.MANAGER_ROLE();  // bytes32 hash -
COSMIX_MANAGER_ROLE
  const adminRoleHash = await ft.DEFAULT_ADMIN_ROLE(); // bytes32 hash -
DEFAULT_ADMIN_ROLE

  const isMinter = await ft.hasRole(minterRoleHash, deployer.address);
  const isManager = await ft.hasRole(managerRoleHash, deployer.address);
  const isAdmin = await ft.hasRole(adminRoleHash, deployer.address);

  console.log("Deployer address:", deployer.address);
  console.log("Deployer is COSMIX_MINTER_ROLE?  ", isMinter ? "☑" : "❌");
  console.log("Deployer is COSMIX_MANAGER_ROLE? ", isManager ? "☑" : "❌");
  console.log("Deployer is DEFAULT_ADMIN_ROLE?  ", isAdmin ? "☑" : "❌");

  // Se vuoi vedere i valori hash effettivi
  console.log("Hash COSMIX_MINTER_ROLE:", minterRoleHash);
  console.log("Hash COSMIX_MANAGER_ROLE:", managerRoleHash);
  console.log("Hash DEFAULT_ADMIN_ROLE:", adminRoleHash);

  // Se vuoi forzare uno stop in caso di permesso mancante
  if (!isManager) {
    throw new Error("Deployer does NOT have COSMIX_MANAGER_ROLE! Cannot grant
roles on FT.");
    }

  try {
    // Helper: grant role se mancante, con fee EIP-1559 e retry una volta
    async function grantIfMissing(contract, role, account, label) {
      const already = await contract.hasRole(role, account);
      if (already) {
        console.log(`ⓘ ${label}: ruolo già presente su ${account}`);
        return true;
      }
      let gasLimit;
      try {
        gasLimit = await contract.estimateGas.grantRole(role, account);
```

```
      } catch {
        gasLimit = 200000n;
      }
      const fee = await ethers.provider.getFeeData();
      let maxFeePerGas = fee.maxFeePerGas || ethers.parseUnits("0.05", "gwei");
      let maxPriorityFeePerGas = fee.maxPriorityFeePerGas ||
ethers.parseUnits("0.02", "gwei");
      const tx = await contract.grantRole(role, account, { gasLimit,
maxFeePerGas, maxPriorityFeePerGas });
      const rcpt = await tx.wait();
      const ok = await contract.hasRole(role, account);
      if (ok) {
        console.log(`☑ ${label}: ruolo assegnato (tx: ${tx.hash}, status:
${rcpt.status})`);
        return true;
      }
      // Retry con bump fee
      const fee2 = await ethers.provider.getFeeData();
      maxFeePerGas = (fee2.maxFeePerGas || maxFeePerGas) +
ethers.parseUnits("0.02", "gwei");
      maxPriorityFeePerGas = (fee2.maxPriorityFeePerGas || maxPriorityFeePerGas)
+ ethers.parseUnits("0.01", "gwei");
      const tx2 = await contract.grantRole(role, account, { gasLimit,
maxFeePerGas, maxPriorityFeePerGas });
      const rcpt2 = await tx2.wait();
      const ok2 = await contract.hasRole(role, account);
      if (!ok2) throw new Error(`${label}: ruolo non assegnato dopo retry (tx:
${tx2.hash})`);
      console.log(`☑ ${label}: ruolo assegnato al retry (tx: ${tx2.hash},
status: ${rcpt2.status})`);
      return true;
    }
    // Se sei su localhost, usa esplicitamente il primo account generato da
Hardhat Node
    const isLocal = process.env.HARDHAT_NETWORK === 'localhost';
    let deployer;
    if (isLocal) {
      const localAccounts = await ethers.getSigners();
      deployer = localAccounts[0];
      console.log("📍 Deployer (localhost):", deployer.address);
    } else {
      [deployer] = await ethers.getSigners();
      console.log("📍 Deployer:", deployer.address);
    }
      // Determina l'explorer in base alla rete
  let explorerUrl = "https://basescan.org/address/";
  let explorerName = "BaseScan";
  let tokenName = "COSMIX";
  if (process.env.HARDHAT_NETWORK === "polygon" || (typeof hre !== "undefined"
&& hre.network && hre.network.name === "polygon")) {
        explorerUrl = "https://polygonscan.com/address/";
        explorerName = "Polygonscan";
      } else if (process.env.HARDHAT_NETWORK === "localhost") {
        explorerUrl = "http://localhost:8545/address/";
```

```
        explorerName = "Localhost";
      }

    const balance = await deployer.provider.getBalance(deployer.address);
    console.log("💰 Balance:", ethers.formatEther(balance), "ETH");

    // 1) Prova a riusare un Orchestrator esistente PRIMA di decidere NFT/FT
    const Orchestrator = await
ethers.getContractFactory("OceanMangaOrchestrator");
    console.log("[DEBUG] OceanMangaOrchestrator bytecode length:",
Orchestrator.bytecode.length / 2, "bytes");
    let orchestrator, orchestratorAddress;
    let routerAddress, swapperAddress;
    let swapper;

    if (process.env.ORCHESTRATOR_ADDRESS) {
      orchestratorAddress = process.env.ORCHESTRATOR_ADDRESS;
      orchestrator = await Orchestrator.attach(orchestratorAddress);
      console.log("☑ Orchestrator già deployato (ENV):", orchestratorAddress);
    } else if (fs.existsSync('last-orchestrator.json')) {
      try {
        const prev = JSON.parse(fs.readFileSync('last-orchestrator.json'));
        if (prev && prev.orchestrator) {
          orchestratorAddress = prev.orchestrator;
          orchestrator = await Orchestrator.attach(orchestratorAddress);
          console.log("♻ Orchestrator recuperato da last-orchestrator.json:",
orchestratorAddress);
        }
      } catch (e) {
        console.warn("[WARN] Impossibile leggere last-orchestrator.json. Procedo
senza orchestrator.");
      }
    }

    // 2) Se ho un Orchestrator, preferisco i suoi indirizzi NFT/FT per evitare
mismatch
    let nftAddress, ftAddress, nft, ft;
    if (orchestratorAddress) {
      try {
        const orchNFT = await orchestrator.oceanMangaNFT();
        const orchFT = await orchestrator.lunaComicsFT();
        if (orchNFT && orchNFT !== ethers.ZeroAddress && orchFT && orchFT !==
ethers.ZeroAddress) {
          nftAddress = orchNFT;
          ftAddress = orchFT;
          nft = await
ethers.getContractAt("contracts/nft/OceanMangaNFT.sol:OceanMangaNFT",
nftAddress);
          ft = await ethers.getContractAt("CosmixProtocolToken", ftAddress);
          console.log("☑ Reusing NFT from Orchestrator:", nftAddress);
          console.log("☑ Reusing FT from Orchestrator:", ftAddress);
        }
      } catch {}
    }
```

```
    // 3) Se non ho ancora NFT/FT, usa ENV oppure deploya nuovi
    if (!nftAddress || !ftAddress) {
      if (process.env.NFT_ADDRESS && process.env.FT_ADDRESS) {
        nftAddress = process.env.NFT_ADDRESS;
        ftAddress = process.env.FT_ADDRESS;
        nft = await
ethers.getContractAt("contracts/nft/OceanMangaNFT.sol:OceanMangaNFT",
nftAddress);
        ft = await ethers.getContractAt("CosmixProtocolToken", ftAddress);
        console.log("☑ NFT proxy già deployato:", nftAddress);
        console.log("☑ FT proxy già deployato:", ftAddress);
      } else {
        // Deploy NFT
        console.log("\n🎨 Deploying OceanMangaNFT...");
        const NFT = await
ethers.getContractFactory("contracts/nft/OceanMangaNFT.sol:OceanMangaNFT");
        console.log("🔧 Deploying NFT as UUPS proxy and initializing with
deployer as admin...");
        nft = await upgrades.deployProxy(
          NFT,
          [
            deployer.address, // admin
            "", // initialURI
            deployer.address, // royaltyReceiver (treasury)
            0 // royalty fee numerator (bps)
          ],
          { initializer: 'initialize', kind: 'uups' }
        );
        await nft.waitForDeployment();
        nftAddress = await nft.getAddress();
        console.log("☑ NFT proxy deployed & initialized:", nftAddress);
        // DEBUG SNIPPET: Diagnosi ruoli e permessi deployer su OceanMangaNFT
(NFT)
     console.log("== DEBUG: Diagnostica ruoli su NFT ==");

     const nftMinterRoleHash = await nft.MINTER_ROLE();          // bytes32 hash
- MINTER_ROLE
     const nftAdminRoleHash  = await nft.DEFAULT_ADMIN_ROLE();   // bytes32 hash
- DEFAULT_ADMIN_ROLE

     const isNftMinter = await nft.hasRole(nftMinterRoleHash, deployer.address);
     const isNftAdmin  = await nft.hasRole(nftAdminRoleHash, deployer.address);

     console.log("Deployer address:", deployer.address);
     console.log("Deployer is NFT MINTER_ROLE?  ", isNftMinter ? "☑" : "❌");
     console.log("Deployer is NFT DEFAULT_ADMIN_ROLE? ", isNftAdmin ? "☑" :
"❌");

     console.log("Hash NFT MINTER_ROLE:", nftMinterRoleHash);
     console.log("Hash NFT DEFAULT_ADMIN_ROLE:", nftAdminRoleHash);

     if (!isNftAdmin) {
      throw new Error("Deployer does NOT have DEFAULT_ADMIN_ROLE on NFT! Cannot
```

```
grant roles on NFT.");
     }
        // Deploy FT
        console.log("\n🔘 Deploying CosmicsProtocolToken...");
        const FT = await ethers.getContractFactory("CosmixProtocolToken");
        console.log("🔧 Deploying FT as UUPS proxy e inizializzando con nome
'Cosmics' e simbolo 'COSMICS'...");
        const initialSupply = ethers.parseUnits("1000000", 18);
        ft = await upgrades.deployProxy(
          FT,
          [
            deployer.address,
            initialSupply,
            deployer.address // treasury
          ],
          { initializer: 'initialize', kind: 'uups' }
        );
        await ft.waitForDeployment();
        ftAddress = await ft.getAddress();
        console.log("☑ FT proxy deployed & initialized:", ftAddress);
      }
    }

    console.log("\n🔄 Deploying NEW Orchestrator with REAL addresses...");

    // REAL WALLET ADDRESSES
    const creatorAddress = deployer.address; // Developer/Creator
    const charityAddress = "0xf84eb8B95407f04Dee8fF9acaC0e0AC7231bAb2A"; //
Caritas International

    console.log("📍 Creator wallet:", creatorAddress);
    console.log("📍 Charity wallet (Caritas):", charityAddress);

    // A questo punto, se orchestratorAddress non esiste, allora deploya un
nuovo Orchestrator con gli indirizzi NFT/FT attuali
    if (!orchestratorAddress) {
      try {
        // Usa fee EIP-1559 della rete (fallback a 0.1 gwei se non disponibili)
        const fee = await ethers.provider.getFeeData();
        const maxFeePerGas = fee.maxFeePerGas || ethers.parseUnits("0.1",
"gwei");
        const maxPriorityFeePerGas = fee.maxPriorityFeePerGas ||
ethers.parseUnits("0.01", "gwei");
        console.log(`[CHECK] maxFeePerGas: ${ethers.formatUnits(maxFeePerGas,
"gwei")} gwei`);
        console.log(`[CHECK] maxPriorityFeePerGas:
${ethers.formatUnits(maxPriorityFeePerGas, "gwei")} gwei`);
        console.log("[DEBUG] Deploying Orchestrator (EIP-1559 fees)");

        // Non forziamo gasLimit: lasciamo stimare al nodo
        orchestrator = await Orchestrator.deploy(
          nftAddress,
          ftAddress,
          creatorAddress,
```

```
          charityAddress,
          { maxFeePerGas, maxPriorityFeePerGas }
        );

        const depTx = orchestrator.deploymentTransaction &&
orchestrator.deploymentTransaction();
        if (depTx && depTx.hash) console.log("[DEBUG] Deploy tx hash:",
depTx.hash);
        console.log("[DEBUG] Deploy transaction sent, waiting for
confirmation...");
        await orchestrator.waitForDeployment();
        orchestratorAddress = await orchestrator.getAddress();
        console.log("☑ New Orchestrator deployed:", orchestratorAddress);
        // Persisti l'indirizzo per evitare redeploy futuri
        fs.writeFileSync('last-orchestrator.json', JSON.stringify({
orchestrator: orchestratorAddress }, null, 2));
        console.log("[PERSIST] Saved orchestrator address to
last-orchestrator.json");
      } catch (e) {
        console.error("[ERROR] Orchestrator deploy failed:", e);
        // Log aggiuntivo su fee data per diagnosi
        try {
          const fee2 = await ethers.provider.getFeeData();
          console.error("[DEBUG] feeData on error:", {
            maxFeePerGas: fee2.maxFeePerGas && fee2.maxFeePerGas.toString(),
            maxPriorityFeePerGas: fee2.maxPriorityFeePerGas &&
fee2.maxPriorityFeePerGas.toString(),
            gasPrice: fee2.gasPrice && fee2.gasPrice.toString(),
          });
        } catch {}
        throw e;
      }
    }

    console.log("\n🔐 Setting up COMPLETE permissions and roles...");

  // Get all role constants
  let MINTER_ROLE, DEFAULT_ADMIN_ROLE, FT_MINTER_ROLE;
  try {
    MINTER_ROLE = await nft.MINTER_ROLE();
    DEFAULT_ADMIN_ROLE = await nft.DEFAULT_ADMIN_ROLE();
    console.log("[DEBUG] NFT MINTER_ROLE via contract method");
  } catch (e) {
    // fallback: OpenZeppelin default hash
    MINTER_ROLE = ethers.keccak256(ethers.toUtf8Bytes("MINTER_ROLE"));
    DEFAULT_ADMIN_ROLE =
ethers.keccak256(ethers.toUtf8Bytes("DEFAULT_ADMIN_ROLE"));
    console.log("[DEBUG] NFT MINTER_ROLE via keccak fallback");
  }
  try {
    FT_MINTER_ROLE = await ft.MINTER_ROLE();
    console.log("[DEBUG] FT MINTER_ROLE via contract method");
  } catch (e) {
    FT_MINTER_ROLE = ethers.keccak256(ethers.toUtf8Bytes("MINTER_ROLE"));
```

```
    console.log("[DEBUG] FT MINTER_ROLE via keccak fallback (MINTER_ROLE)");
  }

    console.log("🔍 Role addresses:");
    console.log("  MINTER_ROLE:", MINTER_ROLE);
    console.log("  DEFAULT_ADMIN_ROLE:", DEFAULT_ADMIN_ROLE);

    // Verify deployer has admin roles
    console.log("\n🔐 Verifying deployer admin permissions...");
    let nftAdminRole = false, ftAdminRole = false;
    try {
      nftAdminRole = await nft.hasRole(DEFAULT_ADMIN_ROLE, deployer.address);
      console.log("  NFT Admin Role (deployer):", nftAdminRole ? "☑" : "❌");
      if (nftAdminRole) {
        console.log("  NFT DEFAULT_ADMIN_ROLE is held by:", deployer.address);
      }
    } catch (e) {
      console.warn("[WARN] NFT contract does not support hasRole. Skipping admin
check.");
    }
    try {
      ftAdminRole = await ft.hasRole(DEFAULT_ADMIN_ROLE, deployer.address);
      console.log("  FT Admin Role (deployer):", ftAdminRole ? "☑" : "❌");
      if (ftAdminRole) {
        console.log("  FT DEFAULT_ADMIN_ROLE is held by:", deployer.address);
      }
    } catch (e) {
      console.warn("[WARN] FT contract does not support hasRole. Skipping admin
check.");
    }

    // Grant MINTER_ROLE to orchestrator on NFT contract
    console.log("\n🎨 Granting NFT minting permissions...");
    const nftAdminRole2 = await nft.hasRole(DEFAULT_ADMIN_ROLE,
deployer.address);
    if (!nftAdminRole2) throw new Error("Deployer missing DEFAULT_ADMIN_ROLE on
NFT");
    await grantIfMissing(nft, MINTER_ROLE, orchestratorAddress, "NFT MINTER_ROLE
→ orchestrator");
    // Diagnostica ruoli FT (non interrompe se manca MANAGER/MINTER, solo log)
    try {
      if (ft.getRoles) {
        const [hasMinterFtDiag, hasManagerFtDiag, hasAdminFtDiag] = await
ft.getRoles(orchestratorAddress);
        console.log(`🔎 FT Roles (diagnostic) on orchestrator:
MINTER=${hasMinterFtDiag}, MANAGER=${hasManagerFtDiag},
ADMIN=${hasAdminFtDiag}`);
      }
    } catch (diagErr) {
      console.warn("[WARN] FT getRoles diagnostic failed:", diagErr.message);
    }

    // Grant MINTER_ROLE to orchestrator on FT contract (usa admin di default)
    console.log("🪙 Granting FT minting permissions...");
```

```
    await grantIfMissing(ft, FT_MINTER_ROLE, orchestratorAddress, "FT
MINTER_ROLE → orchestrator");
// Verify orchestrator has minting permissions
    console.log("\n🔍 Verifying orchestrator permissions...");
    const nftMinterRole = await nft.hasRole(MINTER_ROLE, orchestratorAddress);
  const ftMinterRole = await ft.hasRole(FT_MINTER_ROLE, orchestratorAddress);
    console.log("  NFT Minter Role (orchestrator):", nftMinterRole ? "☑" :
"❌");
    console.log("  FT Minter Role (orchestrator):", ftMinterRole ? "☑" : "❌");

    // Verify orchestrator configuration
    console.log("\n🔍 Verifying orchestrator configuration...");
    const orchNFT = await orchestrator.oceanMangaNFT();
    const orchFT = await orchestrator.lunaComicsFT();
    const orchCreator = await orchestrator.creator();
    const orchCharity = await orchestrator.charityFund();

    console.log("  Orchestrator NFT address:", orchNFT === nftAddress ? "☑" :
"❌", orchNFT);
    console.log("  Orchestrator FT address:", orchFT === ftAddress ? "☑" :
"❌", orchFT);
    console.log("  Creator address:", orchCreator === creatorAddress ? "☑" :
"❌", orchCreator);
    console.log("  Charity address:", orchCharity === charityAddress ? "☑" :
"❌", orchCharity);

    if (!nftMinterRole || !ftMinterRole) {
      throw new Error("❌ Orchestrator missing required minting permissions!");
    }

    console.log("\n🎯 ALL PERMISSIONS CONFIGURED CORRECTLY!");

    console.log("\n🔄 Deploying Token Router & Swapper...");

    // Evita di ridistribuire l'orchestrator: usa quello già presente
    console.log("♻️ Using orchestrator:", orchestratorAddress);

    // Blocca il deploy degli altri contratti finché l'Orchestrator non è
deployato
    if (!orchestratorAddress) {
      console.error("❌ Deploy Orchestrator fallito. Blocca il deploy degli
altri contratti.");
      return;
    }
    // Solo dopo l'ok del deploy dell'Orchestrator, si potranno deployare gli
altri contratti
  // Helper: controllo fondi prima dei deploy costosi
  async function canAffordDeploy(factory, args, label) {
    try {
      const gasEstimate = await factory.estimateGas.deploy(...args);
      const fee = await ethers.provider.getFeeData();
      const price = fee.maxFeePerGas || fee.gasPrice || ethers.parseUnits("1",
"gwei");
      const cost = gasEstimate * price;
```

```
      const bal = await deployer.provider.getBalance(deployer.address);
      const margin = cost + (cost / 10n); // +10%
      if (bal < margin) {
        console.warn(`⚠️ Insufficient funds to deploy ${label}. Needed
~${ethers.formatEther(margin)} ETH, balance ${ethers.formatEther(bal)} ETH.`);
        return { ok: false, needed: margin, balance: bal };
      }
      return { ok: true };
    } catch (e) {
      console.warn(`[WARN] Unable to estimate gas for ${label}:`, e.message);
      return { ok: true };
    }
  }

  // Deploy TokenRouter
  console.log("\n🔄 Deploying TokenRouter...");
  const TokenRouter = await ethers.getContractFactory("TokenRouter");
  const affordRouter = await canAffordDeploy(TokenRouter, [orchestratorAddress,
ftAddress], "TokenRouter");
  if (!affordRouter.ok) {
    console.log("⏭️ Skipping TokenRouter deployment due to low balance. Top-up
and rerun this step.");
  } else {
    const fee = await ethers.provider.getFeeData();
    const router = await TokenRouter.deploy(orchestratorAddress, ftAddress, {
      maxFeePerGas: fee.maxFeePerGas || undefined,
      maxPriorityFeePerGas: fee.maxPriorityFeePerGas || undefined,
    });
    await router.waitForDeployment();
    routerAddress = await router.getAddress();
    console.log("☑️ TokenRouter deployed:", routerAddress);
  }

    // Deploy Swapper
    console.log("\n🔄 Deploying AdvancedSwapper...");
    const Swapper = await ethers.getContractFactory("AdvancedSwapper");
    const affordSwapper = await canAffordDeploy(Swapper, [orchestratorAddress,
ftAddress], "AdvancedSwapper");
    if (!affordSwapper.ok) {
      console.log("⏭️ Skipping AdvancedSwapper deployment due to low balance.
Top-up and rerun this step.");
    } else {
      const feeS = await ethers.provider.getFeeData();
      swapper = await Swapper.deploy(orchestratorAddress, ftAddress, {
        maxFeePerGas: feeS.maxFeePerGas || undefined,
        maxPriorityFeePerGas: feeS.maxPriorityFeePerGas || undefined,
      });
      await swapper.waitForDeployment();
      swapperAddress = await swapper.getAddress();
      console.log("☑️ AdvancedSwapper deployed:", swapperAddress);
    }

  // Grant MINTER_ROLE to swapper (reverse swaps) – usa helper uniforme
  console.log("🔐 Granting swapper minting permissions...");
```

```
  if (swapperAddress) {
    await grantIfMissing(ft, FT_MINTER_ROLE, swapperAddress, "FT MINTER_ROLE → swapper");
  } else {
    console.log("ℹ️ Skipping swapper role grant (no swapper deployed)");
  }

    // Integra nuovi token nello swapper
    console.log("🔗 Adding supported tokens to swapper...");
    // ALGORAND
    await swapper.addSupportedToken("0x3a51f2a377ea8b55faf3c671138a00503b031af3", "1000000000000000", "AGORAND", 18);
    // Arbitrum (ARB)
    await swapper.addSupportedToken("0x912CE59144191C1204E64559FE8253a0e49E6548", "1000000000000000000", "ARB", 18);
    // BAL
    await swapper.addSupportedToken("0xBA12222222228d8Ba445958a75a0704d566BF2C8", "1000000000000000000", "BAL", 18);
    // BIO
    await swapper.addSupportedToken("0x226a2fa2556c48245e57cd1cba4c6c9e67077dd2", "1000000000000000", "BIO", 18);
    // cbETH
    await swapper.addSupportedToken("0x2Ae3F1Ec7F1F5012CFEab0185bfc7aa3cf0DEc22", "1000000000000000000", "cbETH", 18);
    // Chainlink (LINK)
    await swapper.addSupportedToken("0x514910771AF9Ca656af840dff83E8264EcF986CA", "1000000000000000000", "LINK", 18);
    // Compound USDC (cUSDCv3)
    await swapper.addSupportedToken("0xb125e6687d4313864e53df431d5425969c15eb2f", "10000000000000000", "cUSDCv3", 18);
    // DAI
    await swapper.addSupportedToken("0x50c5725949A6F0c72E6C4a641F24049A917DB0Cb", "1000000000000000", "DAI", 18);
    // DepressionStop Token (DST) su Polygon
    await swapper.addSupportedToken("0xAf783f67a83754b5989256fA180534232dF83a0a", "1000000000000000000", "DST", 18);
    // ETH Bridge
    await swapper.addSupportedToken("0x4200000000000000000000000000000000000006", "1000000000000000000", "ETHBRIDGE", 18);
    // Ethereum Universal Token
    await swapper.addSupportedToken("0x1cff25b095cf6595afabe35dd7e5348666e57c11", "1000000000000000000", "ETHU", 18);
    // LINK
```

```
    await
swapper.addSupportedToken("0x514910771AF9Ca656af840dff83E8264EcF986CA",
"1000000000000000000", "LINK", 18);
    // MATIC Universal Token
    await
swapper.addSupportedToken("0xe868c3d83ec287c01bcb533a33d197d9bfa79dad",
"1000000000000000000", "MATICU", 18);
    // OOE
    await
swapper.addSupportedToken("0x6cbb2598881940d08d5ea3fa8f557e02996e1031",
"1000000000000000", "OOE", 18);
    // OP
    await
swapper.addSupportedToken("0x4200000000000000000000000000000000000042",
"1000000000000000000", "OP", 18);
    // Price Oracle Sentinel
    await
swapper.addSupportedToken("0xD228edaA3BD33D604f5561e187782aD9D5B65571",
"1000000000000000", "SENTINEL", 18);
    // protocol token
    await
swapper.addSupportedToken("0x1cff25b095cf6595afabe35dd7e5348666e57c11",
"1000000000000000", "TOKEN", 18);
    // SLD Solidary Token
    await
swapper.addSupportedToken("0x18794e4168dc77f0ee3963ef3c3b4460b33cfb5b",
"1000000000000000", "SLDY", 18); // Sostituisci con l'indirizzo reale di SLDY
    // SOL Solidary Music
    await
swapper.addSupportedToken("0x321974e059c54d009eb57d77ea903d70e6edbef2",
"1000000000000000", "SOLMUS", 18);
    // uDogeToken
    await
swapper.addSupportedToken("0x12e96c2bfea6e835cf8dd38a5834fa61cf723736",
"1000000000000000", "uDOGE", 18);
    // USDC
    await
swapper.addSupportedToken("0x833589fCD6eDb6E08f4c7C32D4f71b54bdA02913",
"1000000000000", "USDC", 6);
    // USDC Bridge
    await
swapper.addSupportedToken("0x46ae9BaB8CEA96610807a275EBD36f8e916b5C61",
"1000000000000", "USDCBRIDGE", 6);
    // USDC MATIC Wrapper
    await
swapper.addSupportedToken("0xD89ea85ee5dD2027dbC29Fbc198DC197D44c3d70",
"1000000000000", "USDCMATIC", 6);
    // USOL
    await
swapper.addSupportedToken("0x9b8df6e244526ab5f6e6400d331db28c8fdddb55",
"1000000000000000", "USOL", 18);
    console.log("☑ Tutti i token integrati nello swapper!");

    // Save deployment info
```

```
    const deploymentInfo = {
      timestamp: new Date().toISOString(),
      network: "base",
      deployer: deployer.address,
      wallets: {
        deployer: deployer.address,
        creator: creatorAddress,
        charity: charityAddress,
        charityAlias: "caritasinternational.cb.id"
      },
      contracts: {
        OceanMangaNFT: nftAddress,
        LunaComicsFT: ftAddress,
        OceanMangaOrchestrator: orchestratorAddress,
  TokenRouter: routerAddress,
      },
      supportedSwapTokens: {
        WETH: "0x4200000000000000000000000000000000000006",
        USDC: "0x833589fCD6eDb6E08f4c7C32D4f71b54bdA02913",
        DAI: "0x50c5725949A6F0c72E6C4a641F24049A917DB0Cb",
        cbETH: "0x2Ae3F1Ec7F1F5012CFEab0185bfc7aa3cf0DEc22"
      },
      feeDistribution: {
        creatorShare: "2.5%",
        charityShare: "2.5%",
        totalFees: "5%"
      },
      gasUsed: "Minimal",
      status: "READY_FOR_MINTING"
    };

    fs.writeFileSync('BASE_COMPLETE_ECOSYSTEM.json',
JSON.stringify(deploymentInfo, null, 2));

    // Persist orchestrator address
fs.writeFileSync('last-orchestrator.json', JSON.stringify({ orchestrator:
orchestratorAddress }, null, 2));
console.log("[PERSIST] Saved orchestrator address to last-orchestrator.json");

    console.log("\n🎉 COMPLETE ECOSYSTEM + SWAPPER DEPLOYED ON BASE!");
    console.log("═══════════════════════════════════════════════");
    console.log("📍 NFT Contract:", nftAddress);
    console.log("📍 FT Contract:", ftAddress);
    console.log("📍 Orchestrator:", orchestratorAddress);
    console.log("📍 Token Router:", routerAddress);
    console.log("📍 Advanced Swapper:", swapperAddress);
    console.log("💾 Config saved to BASE_COMPLETE_ECOSYSTEM.json");

    console.log("\n🚀 ECOSYSTEM READY FOR REAL MINTING!");
    console.log("═══════════════════════════════════");
    console.log("☑ All contracts deployed on Base network");
    console.log("☑ Orchestrator has MINTER_ROLE on both NFT/FT");
    console.log("☑ Deployer retains DEFAULT_ADMIN_ROLE for management");
    console.log("☑ Ultra-low gas fees (~$0.0002 per mint)");
```

```
    console.log("☑ Real blockchain minting now possibile!");

    console.log("\n🔐 SECURITY VERIFICATION:");
    console.log("═══════════════════════════");
    console.log("• Deployer =", deployer.address);
    console.log("• Has admin control over NFT contract ☑");
    console.log("• Has admin control over FT contract ☑");
    console.log("• Orchestrator can mint NFTs ☑");
    console.log("• Orchestrator can mint FTs ☑");
    console.log("• Creator fees (2.5%) go to:", creatorAddress, "☑");
    console.log("• Charity fees (2.5%) go to Caritas:", charityAddress, "☑");

    console.log("\n📱 UPDATE REACT APP:");
    console.log("═══════════════════════");
    console.log(`Update .env file:`);
    console.log(`VITE_ORCHESTRATOR_CONTRACT_ADDRESS=${orchestratorAddress}`);

    console.log("\n  TOKEN SWAPPING FEATURES:");
    console.log("═══════════════════════════");
    console.log(`☑ ${tokenName} ↔ WETH (Wrapped Ethereum)`);
    console.log(`☑ ${tokenName} ↔ USDC (USD Coin)`);
    console.log(`☑ ${tokenName} ↔ DAI (Maker DAI)`);
    console.log(`☑ ${tokenName} ↔ cbETH (Coinbase ETH)`);
    console.log("💰 Conversion fee: 0.30%");
    console.log("🌓 Slippage protection enabled");
    console.log("🔄 Upgradeable proxy pattern");

    console.log("\n💡 SUGGESTED ADDITIONAL TOKENS:");
    console.log("═══════════════════════════════");
    console.log("• COMP (Compound) - DeFi governance");
    console.log("• AAVE - Lending protocol");
    console.log("• UNI (Uniswap) - DEX token");
    console.log("• LINK (Chainlink) - Oracle network");
    console.log("• WBTC - Wrapped Bitcoin");
    console.log("• OP (Optimism) - Layer 2 token");

    console.log("\n 🔗 BaseScan Links:");
    console.log("═══════════════════");
    console.log(`\n🔗 ${explorerName} Links:`);
    console.log("═══════════════════");
    console.log(`NFT: ${explorerUrl}${nftAddress}`);
    console.log(`FT: ${explorerUrl}${ftAddress}`);
    console.log(`Orchestrator: ${explorerUrl}${orchestratorAddress}`);
    console.log(`Router: ${explorerUrl}${routerAddress}`);
    console.log(`Swapper: ${explorerUrl}${swapperAddress}`);

  } catch (error) {
    console.error("❌ Deploy failed:", error.message || error);
    console.error("Error details:", error.stack || error);
    process.exitCode = 1;
  }
}

// SUGGERIMENTO: Per testare senza bruciare gas, usa la rete locale o Sepolia
```

```
// Esempio per la rete locale:
// npx hardhat node
// npx hardhat run scripts/deploy-minimal-base-ecosystem.cjs --network localhost
// Esempio per Sepolia:
// npx hardhat run scripts/deploy-minimal-base-ecosystem.cjs --network sepolia

main().catch((error) => {
  console.error(error);
  process.exitCode = 1;
});
```